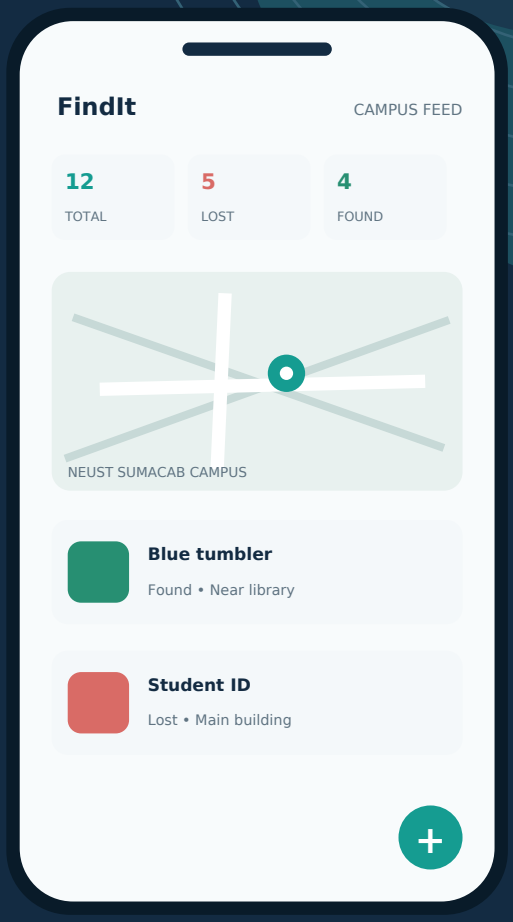
NEUST / CICT
COLLEGE OF INFORMATION AND COMMUNICATIONS TECHNOLOGY

IT-WS07 • MOBILE APPLICATION TECHNOLOGY

FindIt

Campus Lost & Found Mobile Application

FINAL CASE STUDY / SOFT COPY DOCUMENTATION



PROJECT SUMMARY

A centralized, map-enabled registry for reporting and tracking lost or found campus belongings.

PREPARED BY

John Carlo DG. Lemence	SUM2023-01395
Chris Karl M. Lacanilao	SUM2023-01362
Manny N. Agulto	SUM2023-01888
Reyven B. Calara	SUM2023-02252

PROJECT LINKS

GitHub repository

Android APK v1.0

Video presentation

01 / FINDIT CASE STUDY

Project overview

A focused campus solution that helps students and staff report, locate, and track misplaced belongings.

PROJECT FindIt: Campus Lost & Found Mobile Application • IT-WS07 Mobile Application Technology • NEUST CICT

The problem on campus

Lost and found belongings are often handled through word of mouth, physical bulletin boards, or scattered social media posts. This makes it difficult to identify an item, find its exact location, or know whether it has already been returned.

01 / LOCATION GAPS

Vague notes such as “near the canteen” do not show the exact vicinity, leaving the owner to search without a precise reference.

02 / NO VISUAL CHECK

Text-only descriptions such as “blue umbrella” can be ambiguous and make ownership harder to verify.

03 / UNCLEAR STATUS

Without a shared status, users may not know whether an item is still missing or has already been returned.

Intended users

Students and faculty
Campus occupants who lose personal items or find unattended belongings.

Campus security and department staff
Personnel who receive found items, record them, attach visual evidence, and mark them as claimed after return.

Proposed solution

FindIt is a cross-platform mobile application built with Ionic, Angular Standalone, and Capacitor. It provides a shared cloud-backed registry where users can publish a lost or found report, attach a photo, pin a location on a campus map, search existing reports, and update an item’s status.

CORE VALUE One place to report an item, see its photo and map location, and follow its status from Open to Claimed.

02 / FINDIT CASE STUDY

Application features

The application is organized around three connected views: browse reports, submit a report, and manage an item.

The three primary views

01 / HOME & ITEMS LIST

Quick overview
Four live counters show Total, Lost, Found, and Claimed items.

Find a report
Search names, descriptions, and location notes; filter by All, Lost, Found, or Claimed.

Campus map
Collapsible OpenStreetMap view centered on NEUST Sumacab Campus.

02 / POST ITEM

Report details
Choose Lost or Found, then enter the item name, description, and contact information.

Attach a photo
Use the device camera or photo library and preview or remove the image.

Set location
Use Locate Me or tap the Leaflet map to place or move a pin.

03 / ITEM DETAIL

Verify the report
View the item photo, description, contact details, status, and a focused map pin.

Update status
Mark an open item as Claimed.

Remove a report
Delete obsolete, duplicate, or test entries after confirmation.

Typical user flow

1	2	3	4	5
Browse Open the campus feed	Report Enter item details	Attach Add photo and pin	Publish Send report to API	Resolve Mark as Claimed

USER FEEDBACK The interface provides loading, empty, and error states. A Retry action is shown when the network request fails; form validation warns users when required information or a location is missing.

The Home page also uses reusable item cards and a floating action button to open the reporting form. Reactive Angular signals keep the counters and visible list aligned with the current report data.

03 / FINDIT CASE STUDY

Technical implementation

The app combines native device capabilities, an interactive map, cloud REST endpoints, and local photo caching.

Technology stack

Layer	Technology	Role in FindIt
Mobile UI	Ionic Framework v9	Mobile-first interface components for pages, cards, forms, filters, and navigation.
Application logic	Angular Standalone	Standalone components, dependency injection, and signal-based reactive state.
Device capabilities	Capacitor v8	Camera, Geolocation, and Haptics plugins connect the app to device features.
Map interface	Leaflet v1.9 + OpenStreetMap	Interactive campus map, GPS positioning, and report location markers.
Data access	Angular HttpClient + MockAPI.io	HTTPS REST requests and shared report records.
Image hosting	ImgBB Cloud CDN	Stores uploaded report images and returns direct HTTPS image URLs.
Local cache	Browser localStorage	Keeps a local photo copy by item ID for quick retrieval on that device.

Mobile capabilities used

Capability	Implementation	Purpose
Camera	Capacitor Camera	Capture a photo or select one from the device gallery.
Geolocation	Capacitor Geolocation	Read current coordinates and center the post form map.
Haptics	Capacitor Haptics	Provide tactile success, warning, and error feedback.

Data and image flow

CLIENT APP
Home, Post Item, and Item Detail views. The ItemService manages report state and requests; Leaflet displays map data.

HYBRID STORAGE
MockAPI stores report data and image URLs. ImgBB hosts uploaded images; localStorage keeps a device-side photo cache.

DEVICE SERVICES
Capacitor Camera, Geolocation, and Haptics supply photos, coordinates, and tactile feedback.

REST API operations

Method	Operation in FindIt	Result
GET	Fetch the report list	Loads shared reports into the app state.
POST	Create a lost or found report	Adds a report to the cloud registry.
PUT	Mark an item as Claimed	Updates the selected report status.
DELETE	Remove a report	Deletes the report and clears its local photo cache.

Before upload, the report image is prepared at a target width of 800 px and sent to ImgBB. FindIt stores the returned image URL in the MockAPI record while keeping a local photo copy keyed by the item ID. This keeps the API record compact and supports quick local photo display.

04 / FINDIT CASE STUDY

Testing and evaluation

Core reporting, location, media, filtering, and status-management flows were exercised on the application.

Functional test results

11 documented scenarios • all passed

#	Feature / task	Expected result	Observed result	Status
1	Open app / GET	Reports load; map renders.	Seed items loaded and campus map rendered.	PASS
2	API unavailable	Error state and Retry; no crash.	Error message, Retry action, and error haptic shown.	PASS
3	Lost / Found / Claimed filters	List shows selected status.	Segment changes updated the list immediately.	PASS
4	Submit empty required fields	Validation prevents submission.	Error shown; warning haptic; no POST sent.	PASS
5	Locate Me with permission	Map centers on device GPS.	Map centered on GPS and placed a marker.	PASS
6	Locate Me permission denied	Error shown; map remains usable.	Error banner shown; manual map pin still available.	PASS
7	Attach camera / gallery photo	Image preview appears.	Photo previewed; remove action worked.	PASS
8	Submit valid report / POST	Report is saved and displayed.	Success haptic; returned Home; report appeared with photo.	PASS
9	Open item detail	Details, photo, and map marker appear.	Metadata, full photo, and focused map loaded.	PASS
10	Mark as Claimed / PUT	Status updates to Claimed.	Cloud record and UI badge updated.	PASS
11	Delete report / DELETE	Confirmation then removal.	Confirmed deletion; API record removed and list refreshed.	PASS

Automated verification

UNIT TESTS

7 / 7 suites passed
Vitest coverage includes ItemService, ItemDetailPage, PostItemPage, HomePage, and ItemCardComponent, including REST and signal-state behavior.

END-TO-END TESTS

4 / 4 tests passed
Playwright mobile viewport checks the Home screen and filters, campus map, reporting form, and keyword search.

EVALUATION

The recorded checks cover the primary user journey and key failure paths. Usability can be further improved through stronger access control, offline map support, and clearer privacy guidance for uploaded photos.

05 / FINDIT CASE STUDY

Limitations and handoff

Known prototype constraints, recommended next steps, responsible AI disclosure, and installation guidance.

Limitations and recommended improvements

Observed limitation	Recommended improvement
Public, unauthenticated API. The MockAPI endpoint is writable by clients that know its URL.	Add Firebase Authentication or JWT-based access, ideally using official NEUST student email credentials.
External image hosting. Photo upload depends on ImgBB availability; the public prototype should not receive sensitive personal documents.	Use dedicated cloud storage such as Firebase Storage or AWS S3, with clear upload and retention rules.
Online map tiles. Uncached OpenStreetMap areas appear blank without internet access.	Consider local map tile caching through SQLite or IndexedDB for offline browsing.
No server-side duplicate constraint. Similar item titles are allowed intentionally because different users may report similar belongings.	Keep duplicate-title tolerance; consider moderation or report review rather than strict title-based blocking.

Additional protection already implemented: the submit action is locked while a request is in progress to prevent rapid double-submission. The app also advises users not to upload sensitive IDs, grade slips, passwords, or private documents to the public demonstration.

AI usage disclosure

Disclosure area	Details
AI tools consulted	Claude Opus 4.6 for planning and architecture analysis; Gemini 3.8 Flash for implementation scaffolding.
Tasks assisted by AI	Project milestone planning, Angular Standalone route structure, TypeScript scaffolding, and initial Leaflet and Camera integration guidance.
Work reviewed and completed by the group	The group designed the local-first photo strategy, configured MockAPI, tested camera and GPS permissions, haptics and map interaction on Android hardware, built the Android project in Android Studio, and authored the report.
Security and privacy	No personal passwords, private tokens, or proprietary credentials were shared with AI tools. The public API and image CDN are used for an academic prototype; users are advised not to upload sensitive documents.

Installation instructions for the instructor

1	Download FindIt-v1.0.apk from the GitHub Release v1.0 link on the cover.
2	On Android, allow Install unknown apps for the browser or file manager used to open the APK.
3	Open the APK and complete installation. Review the device prompt if Play Protect displays an unrecognized-app warning.
4	Launch FindIt and allow Camera and Location permissions when prompted.
5	Keep Wi-Fi or mobile data enabled while testing; the app uses the live HTTPS MockAPI endpoint and online map tiles.

Project repository, APK download, and recorded presentation are linked on the cover page.